Verification and Validation Report: Software Engineering

Team 13, Speech Buddies
Mazen Youssef
Rawan Mahdi
Luna Aljammal
Kelvin Yu

March 9, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test

[symbols, abbreviations or acronyms – you can reference the SRS tables if needed —SS]

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**

3.1 FR1: Accept Speech Audio via Microphone 1

3.1.1 test-FR1-MIC-1 1

3.1.2 test-FR1-MIC-2 1

3.1.3 test-FR1-MIC-3 1

3.2 FR2: Convert Impaired Speech to Text with ≥80% Accuracy . 2

3.2.1 test-FR2-ASR-1: Dataset Evaluation vs Baseline 2

3.2.2 test-FR2-ASR-2: Common Command Recognition . . . 2

3.3 FR3: Display Transcription for Verification 2

3.3.1 test-FR3-UI-1: Real-Time Display Latency 2

3.3.2 test-FR3-UI-2: Continuous Update & Finalization . . . 3

3.4 FR4: Map Text to Arbitrary Device Commands 3

3.4.1 test-FR4-MAP-1: Canonical Command Set Accuracy . 3

3.4.2 test-FR4-MAP-2: Paraphrase & Synonym Robustness . 3

3.4.3 test-FR4-MAP-3: Ambiguity & Disambiguation 3

3.4.4 test-FR4-MAP-4: Out-of-Domain Commands 4

3.5 FR5: Execute Commands on the Host Device 4

3.5.1 test-FR5-EXEC-1: End-to-End Execution Latency & Reliability 4

3.5.2 test-FR5-EXEC-2: Accessibility/API Fallback Paths . 4

3.5.3 test-FR5-EXEC-3: Safety & Confirmation for Destructive Actions 4

3.6 FR6: Handle Security Software Interference 5

4 Nonfunctional Requirements Evaluation **5**

4.1 Usability 5

4.2 Performance 5

4.3 etc. 5

5 Comparison to Existing Implementation **5**

6 Unit Testing **5**

7 Changes Due to Testing 5

8 Automated Testing 6

9 Trace to Requirements 6

10 Trace to Modules 6

11 Code Coverage Metrics 6

List of Tables

List of Figures

This document ...

3 Functional Requirements Evaluation

This section presents the evaluation results for all functional requirements as defined in the SRS. Each functional requirement is evaluated through the corresponding test cases from Section 4.1 of the V&V Plan. For detailed test specifications (initial state, inputs, expected outputs, and execution methodology), refer to the V&V Plan. This section focuses on test results and traceability.

3.1 FR1: Accept Speech Audio via Microphone

Requirement: The system must capture live speech input from the user through a standard built-in or external microphone, reliably detecting and recording audio from default OS microphone devices across Windows, macOS, iOS, and Android, with a minimum 16 kHz sampling rate.

3.1.1 test-FR1-MIC-1

Summary: Verify microphone permission request and audio capture.
Result: Pass – microphone permissions were requested as expected on Windows, and audio was successfully captured at 16 kHz or higher across all test runs.

3.1.2 test-FR1-MIC-2

Summary: Verify permission handling on iOS and Android platforms.
Result: Pass – permission handling was reliable on both iOS and Android, and microphone access was consistently verified through each platform's device permissions toolbar.

3.1.3 test-FR1-MIC-3

Summary: Verify automatic fallback when audio devices are disconnected during capture.
Result: Pass – when the active audio device was disconnected, the system detected the change and automatically switched to the default device.

3.2 FR2: Convert Impaired Speech to Text with ≥80% Accuracy

Requirement: The system must process captured audio and output textual representation. Transcription accuracy must reach at least 80% WER reduction compared to baseline models, and achieve ≥80% accuracy for common commands.

3.2.1 test-FR2-ASR-1: Dataset Evaluation vs Baseline

Summary: Automated batch evaluation comparing system WER to baseline on impaired-speech test set.
Result: Pass – the system achieved a WER reduction greater than 80% relative to the baseline across the impaired-speech datasets

3.2.2 test-FR2-ASR-2: Common Command Recognition

Summary: Automated evaluation of transcription accuracy for 50+ common command intents.
Result: Pass – transcription accuracy for the 50+ common command intents exceeded 80%, with most commands recognized correctly on the first attempt.

3.3 FR3: Display Transcription for Verification

Requirement: The transcribed text must be displayed in real time on the user's device interface. Every spoken input is displayed within 2 seconds as text on the UI, with at least 95% consistency across trials.

3.3.1 test-FR3-UI-1: Real-Time Display Latency

Summary: Automated measurement of time-to-first-text for 100 utterances across platforms.
Result: Partial – some utterances were displayed within approximately 2 seconds, but most trials experienced additional latency of a couple of seconds before the first text appeared.

3.3.2 test-FR3-UI-2: Continuous Update & Finalization

Summary: Manual verification of incremental text updates and finalization cues during streaming transcription.
Result: Partial – continuous streaming transcription is not yet implemented, but text updates do appear on the UI once recognition completes, so only finalization behavior could be partially observed.

3.4 FR4: Map Text to Arbitrary Device Commands

Requirement: The system must recognize when transcribed text corresponds to a predefined device action and translate it into the appropriate command representation.

3.4.1 test-FR4-MAP-1: Canonical Command Set Accuracy

Summary: Automated evaluation of mapping accuracy for 50 predefined commands.
Result: Pass – commands such as "open Chrome", and "play video" were consistently mapped to the correct internal device actions with high accuracy.

3.4.2 test-FR4-MAP-2: Paraphrase & Synonym Robustness

Summary: Automated evaluation of mapping accuracy with paraphrased and synonymous command phrasings.
Result: Pass – paraphrased commands like "launch Chrome" or "go down the page" were correctly interpreted as their canonical equivalents (e.g., "open Chrome", "scroll down") in the vast majority of cases.

3.4.3 test-FR4-MAP-3: Ambiguity & Disambiguation

Summary: Manual verification of clarification prompts for ambiguous commands.
Result: Pass – when users issued ambiguous commands such as "Try YouTube", the system responded with clarification prompts (e.g., say yes to confirm opening YouTube) and executed the chosen option correctly.

3.4.4 test-FR4-MAP-4: Out-of-Domain Commands

Summary: Manual verification of feedback for unsupported commands.
Result: Pass – for unsupported or out-of-domain requests like "Hack this Website", the system did not attempt execution and instead returned clear feedback that the command was not supported.

3.5 FR5: Execute Commands on the Host Device

Requirement: The system must execute mapped commands through the host device's accessibility framework or APIs, resulting in visible user action.

3.5.1 test-FR5-EXEC-1: End-to-End Execution Latency & Reliability

Summary: Automated measurement of execution latency and success rate for 50-command set across platforms.
Result: Pass – end-to-end command execution completed successfully in roughly 5–10 seconds for most commands, with an overall success rate of about 90%.

3.5.2 test-FR5-EXEC-2: Accessibility/API Fallback Paths

Summary: Verification of fallback behavior when primary API is disabled.
Result: Partial – when the primary API was disabled, commands were not able to be executed and robust fallback mechanisms still need to be implemented.

3.5.3 test-FR5-EXEC-3: Safety & Confirmation for Destructive Actions

Summary: Manual verification of confirmation prompts for destructive commands.
Result: Pass – for destructive prompts such as deleting files or modifying the website, built-in guardrails prevented immediate execution and required explicit confirmation, effectively blocking unsafe actions.

3.6 FR6: Handle Security Software Interference

Requirement: The system must detect when browser, operating system, or antivirus security controls block or restrict automated actions and provide clear feedback to the user, including the affected command and available fallback options. No specific test cases were defined in the V&V Plan for FR6. This requirement was evaluated through scenario-based testing during command execution tests FR5.

4 Nonfunctional Requirements Evaluation

4.1 Usability

4.2 Performance

4.3 etc.

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

7 Changes Due to Testing

[This section should highlight how feedback from the users and from the supervisor (when one exists) shaped the final product. In particular the feedback from the Rev 0 demo to the supervisor (or to potential users) should be highlighted. —SS]

8 Automated Testing

9 Trace to Requirements

10 Trace to Modules

11 Code Coverage Metrics

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)